The scheduler is the tricky part, because Ruby does not distribute a standard scheduler, only an interface that schedulers are expected to implement. Bruno Sutic's website https://github.com/bruno-/fiber_scheduler_list maintains a list of schedulers that are available, and recommends using FiberScheduler[2] in Ruby 3.1 and up.

Understanding Ractors

Ruby 3.0 introduced *ractors*, a Ruby implementation of the Actor pattern for multithreaded behavior. (Experimental support for the feature was originally developed under the name "Guilds.") As of Ruby 3.2, Ractors are still considered experimental.

How Ractors Work

Ractors allow true parallelism within a single Ruby interpreter: each Ractor maintains its own GIL, allowing for potentially better performance. In order for this to work ractors have only limited ability to access variables outside their scope, and can communicate with each other in only specific, pre-defined ways. (Also, if for some reason you are running multiple threads inside a single ractor—probably you shouldn't do this—those threads are subject to the equivalent of a global lock on the ractor and will not run in parallel.)

We think that showing trivial examples of ractor code tends to obfuscate what's going on, so what we're going to do is explain conceptually how ractors work and talk about the API, and then show some code that can actually do a thing.

You can think of a ractor as being a chunk of code that has a single input port and a single output port. Metaphorically, you can think of a room with one door marked "entrance" and one door marked "exit." The entrance door has a potential queue to get in.

You create a ractor with `Ractor.new`, which always takes a block. The block becomes the chunk of code inside our metaphorical room. The `new` method optionally takes an arbitrary number of positional arguments, plus there's an optional keyword argument called `name:` that you should use to give the ractor a unique name. We find it helps to realize that in many or most useful cases, the block will contain a loop of some kind. Once a Ractor is created, the original part of the thread is called the *main* ractor, and can be accessed with `Ractor.main`.

Ractors mostly interact with each other in one of four ways:

- A ractor (including the main thread) can send arguments to a known other ractor. In our metaphor, this is asking somebody to stand in line at the entrance door to a different ractor. The entrance lines are infinite, and the sending call is guaranteed not to block for "if this goes wrong you have much larger problems" meanings of "guaranteed." The API call is `send` and the receiver of the message is the ractor that the message is being sent to, `other_ractor.send(my_args)`. This is similar to the API for fibers.
- A ractor (or the main thread) can take output from a known other ractor. In our metaphor, we are waiting by the exit door for the next value to emerge and grabbing it. The API call here is `take`, as in `new_value = other_ractor.take`, and the `take` call will block waiting for a value to be sent by the other ractor.
- Inside the ractor, the ractor can block waiting for an incoming message. Metaphorically, the ractor is waiting for somebody to show up at the entrance door. The API call here is `Ractor.receive`, and yes, that's a class method of the class `Ractor`.

2. https://github.com/bruno-/fiber_scheduler